

Class and Object in Python

22 July 2026 06:51

What a class and an object are (analogy)

- **Class:** a blueprint, template or recipe that defines attributes (data) and behaviors (methods) for a type of thing.

Student(rollNum, Name, email_id, parent_id)

Parents(parent_id, name, email, address, mobileNum)

RollNum	Name	email-id	parent-id
1	Amy		P001
1	Jhon		P001
3	Jacob		P001
4	Arnon		P001
5	Mitchel		P001

Attributes
or
instance
variable

parent-id	name	email	address	mobileNum
P001	David	david@gmail.com	NS45,	7620512

attributes
or
instance variables

- **Object(instance):** a concrete item created from the blueprint—it holds its own attribute values and can use the class behaviors.
Example analogy: class = blueprint of a car; object = one particular car

Basic class definition and creating objects

- Define a class using the class keyword.
- Create objects by calling the class like a function.

is the constructor (runs when an instance is created). The first parameter of instance method is conventionally named self and refers to the instance.

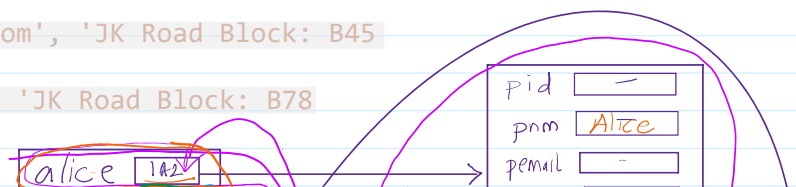
```
class Parent:
    def __init__(self, pid, pnm, pemail, paddr, pmobile):
        self.pid = pid
        self.pnm = pnm
        self.pemail = pemail
        self.paddr = paddr
        self.pmobile = pmobile
    def greet(self):
        return f"Hello {self.pnm} Good Afternoon"
```

Attributes assigned to self (self.var-name) are instance attributes — each object gets its own copy.

main code

```
alice = Parent('p101', 'Alice', 'alice@g.com', 'JK Road Block: B45  
Texsus', '9876523')
bob = Parent('p102', 'Bob', 'alice@g.com', 'JK Road Block: B78  
Texsus', '98765432')
```

this invokes --init--(self,...) method



```
bob = Parent('p102', 'Bob', 'alice@g.com', 'JK Road Block: B/8  
Texsus', '98765432')
```

```
print(alice.greet())  
print(bob.greet())
```

Output:

Hello Alice Good Afternoon
Hello Bob Good Afternoon

